PROGRAMMING 512 ASSIGNMENT

RICHFIELD
richfield.ac.za

Name and Surname: _____

Student ITS No: _____

Qualification: _____ **Year of Study:** _____ **Semester:** _____

Assignment due date: _____ **Date submitted:** _____

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents
Introduction
Main Body (with relevant subheadings)
Conclusion
References

Submit the assignment in PDF format on Moodle. Use the specified cover page provided.
Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:
I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism

Student Signature: _____ **Date:** _____

Case Study: Smart Campus Resource Booking System **100 MARKS**

1. Background

Richfield Graduate Institution is operating across multiple campuses in South Africa and relies heavily on shared academic resources such as computer laboratories, multimedia projectors, seminar rooms, and presentation equipment. These resources are essential for daily teaching, assessments, and academic events.

At present, each campus manages resource bookings manually through email requests, spreadsheets, or paper-based registers. This fragmented approach has led to frequent problems, including double bookings, underutilisation of resources, lack of accountability, and difficulty generating accurate usage reports. Additionally, management has no centralised view of how resources are being used across campuses.

To address these challenges, the institution has requested the development of a **Smart Campus Resource Booking System** using Python. The system must provide a consistent, reliable, and user-friendly solution that supports multiple campuses, enforces booking rules, and stores data persistently for reporting and analysis.

You have been appointed as a junior software developer to design and implement this system based on the requirements below.

2. System Overview

The Smart Campus Resource Booking System must:

- Support multiple campuses using structured data collections such as lists, dictionaries, and sets.
- Provide user authentication and role-based access control using object-oriented design principles.
- Allow users to manage and interact with campus resources through a graphical user interface built using Tkinter.
- Track resource availability, bookings, and usage duration accurately.
- Persist all system data using an SQLite database so that information is retained between program runs.

3. User Roles and Access Control

The system must support **three distinct user roles**, implemented using object-oriented programming concepts such as classes, inheritance, and encapsulation.

3.1 Lecturer

A lecturer represents an academic staff member who books resources for teaching or presentations. Lecturers must be able to:

- Register an account and log into the system using a GUI-based login interface.
- Select a campus and view available resources stored using dictionaries or lists.

- Create a booking for a selected resource, date, and time.
- View their personal booking history retrieved from the database.

3.2 Campus Administrator

A campus administrator manages resources at a specific campus.
Campus administrators must be able to:

- Log in to the system with administrator privileges.
- Add, update, and remove campus resources using appropriate data structures.
- View all bookings associated with their campus.
- Monitor resource availability and usage trends.

3.3 System Operator

A system operator represents institutional management with system-wide access.
System operators must be able to:

- Log in to the system with full access rights.
- View booking and resource data across all campuses.
- Generate comparative reports across campuses.

4. Campus and Resource Operations

4.1 Campus Selection

After a successful login, users must select a campus to work with.

- The system must clearly display the currently active campus in the GUI.
- Campus data must be stored and managed using dictionaries or objects.

4.2 Resource Registration

When a resource is registered:

- It must be associated with a specific campus.
- Each resource must have a unique identifier to prevent duplication.
- Duplicate resource entries must be detected and rejected.

4.3 Resource Booking

When a lecturer books a resource:

- The system must record the booking date, start time, and duration.
- The booking must be linked to both the lecturer and the selected campus.
- The system must prevent overlapping bookings for the same resource.

5. Booking Rules and System Flexibility

5.1 Campus-Specific Booking Policies

Each campus applies its own booking rules.
Requirements:

- The system must include at least **three campuses**, each with different booking constraints.
- Booking rules must be applied automatically based on the selected campus.
- The system must clearly indicate which booking policy is currently in effect.

Examples of booking rule differences may include:

- Maximum booking duration per session
- Restricted booking hours
- Priority access rules

5.2 Design for Future Expansion

The system must be designed so that:

- Booking rules are separated from core system logic using functions or classes.
- New campuses or booking rules can be added with minimal changes to existing code.

6. User Services and Booking Management

6.1 Booking Management

Lecturers must be able to:

- View active bookings.
- Cancel future bookings with updates reflected in the database.
- View past booking history.

6.2 Resource Availability

The system must:

- Display real-time resource availability via the GUI.
- Prevent users from booking unavailable resources.

7. Reporting and Management Information

7.1 Campus-Level Reports

The system must generate reports per campus, including:

- Total number of bookings.
- Most frequently used resources.
- Average booking duration.

7.2 Cross-Campus Comparison

System operators must be able to compare:

- Resource usage across campuses.
- Booking trends and differences resulting from campus-specific rules.

8. Data Storage and Persistence

The system must store and retrieve the following using **SQLite**:

- User accounts and roles.
- Campus and resource information.
- Booking records.

All data must persist between program executions.

9. Deliverables

Students must submit:

1. A complete project folder as a single zipped file (≤ 50 MB).
2. A fully functional Python application demonstrating all required features.

Important:

- Non-functional systems will receive no marks
- Plagiarised or copied solutions are prohibited

10. Notes to Students

- Code must be well-structured, readable, and commented
- GUI design must be user-friendly and intuitive
- Input validation and error handling are essential